



QoreDB

LOCAL-FIRST DATABASE CLIENT

The SQL Performance Cheat Sheet

Find the slow query, read the plan, fix the index. A field guide to the handful of causes behind almost every slow query — and what to type to prove it, on each engine you actually use.

PostgreSQL

MySQL / MariaDB

SQLite

Version-agnostic

Copy-paste ready

01	Measure, don't guess	EXPLAIN on every engine · the five red flags in a plan
02	Indexes that get used	Column order · covering indexes · what silently kills a seek
03	Query patterns	N+1 · keyset pagination · OR · casts · counting
04	Engine-specific tuning	Postgres · InnoDB · SQLite · the 10-minute triage

Every optimisation starts the same way: **get the execution plan**. Intuition about which query is slow is wrong often enough that it isn't worth acting on. The plan tells you what the database actually did — not what you hoped it would do.

■ Get a plan

ENGINE	COMMAND	NOTES
POSTGRES	EXPLAIN (ANALYZE, BUFFERS) <query>	ANALYZE really runs the query . For writes, wrap in BEGIN; ... ROLLBACK;. BUFFERS shows cache hits vs disk reads.
MYSQL	EXPLAIN ANALYZE <query>	MySQL 8.0.18+. Gives actual time and row counts per iterator. Older: EXPLAIN FORMAT=TREE (8.0.16+) or plain EXPLAIN.
SQLITE	EXPLAIN QUERY PLAN <query>	Estimates only — it does not execute. Plain EXPLAIN dumps bytecode; that's not what you want.

ESTIMATED VS ACTUAL

EXPLAIN alone shows what the planner **expects**. EXPLAIN ANALYZE shows what **happened**. The gap between the two is the single most diagnostic number in the whole plan — see "red flag #1" below. Always reach for the ANALYZE form when the query is safe to run.

■ Reading a Postgres plan

Read it **inside-out and bottom-up**: the deepest, most-indented node runs first, and each node feeds its parent.

```
Nested Loop (cost=0.29..8156.2 rows=1 width=64) (actual time=0.04..842.11 rows=4213 loops=1)
-> Seq Scan on orders (cost=0.00..4321.0 rows=1 width=32)
    (actual time=0.02..61.30 rows=4213 loops=1)
    Filter: (status = 'pending'::text)
    Rows Removed by Filter: 495787
-> Index Scan using customers_pkey on customers (actual time=0.18..0.18 rows=1 loops=4213)
    Index Cond: (id = orders.customer_id)
Planning Time: 0.31 ms
Execution Time: 843.02 ms
```

Diagnosis: the planner estimated rows=1 from the orders scan but got 4213. Believing it would loop once, it chose a Nested Loop — which then executed the inner index scan **4 213 times**. It also read half a million rows to throw away 495 787 of them. Two fixes: ANALYZE orders; to correct the estimate, and an index on orders (status) so the scan seeks instead of sweeping.

■ The five red flags

	WHAT YOU SEE	WHAT IT MEANS	FIRST THING TO TRY
1	Estimated rows off from actual by >10×	Stale or missing statistics. The planner is choosing from bad information, so every downstream choice is suspect. Fix this before anything else.	ANALYZE tbl; PG ANALYZE TABLE tbl; MY ANALYZE; LT
2	Seq Scan / type: ALL / SCAN tbl on a big table	Full table read. Correct and fast for small tables or when returning most rows — a problem when you wanted a handful of rows.	Index the filtered column; check the filter is sargable (§2)
3	High Loops=N on an inner node; Rows Removed by Filter large	Work is being repeated per outer row, or read then discarded. Usually a knock-on effect of #1.	Fix stats; index the join/filter column
4	Sort Method: external merge Disk: ... Using filesort / Using temporary	Sorting or grouping spilled out of memory onto disk.	Index matching the ORDER BY; or raise work_mem PG / sort_buffer_size MY
5	Buffers: ... read= large vs hit=	Reading from disk rather than cache. Either the working set doesn't fit in RAM, or you're touching far more data than needed.	Reduce rows touched first; only then size the cache (§4)

■ Join strategies — and when each is right

The planner picks these for you. You can't usually force them (and shouldn't) — but recognising **why** it picked wrong tells you what to fix.

STRATEGY	HOW IT WORKS	RIGHT WHEN	GOES WRONG WHEN
Nested Loop	For each row of the outer input, look up matches in the inner input.	The outer side is small and the inner side has an index on the join key.	The outer side turns out big. Cost is outer × inner-lookup, so a 1-row estimate that's really 100 k rows is catastrophic — red flag #1 .
Hash Join	Build a hash table from the smaller input, then probe it with the larger one.	Joining large, unindexed inputs on equality.	The build side doesn't fit in memory and spills to disk (red flag #4). Equality joins only.
Merge Join	Both inputs are sorted on the join key, then walked in lockstep.	Both inputs are already sorted — typically arriving straight from an index.	It has to sort them first, and the sort spills.

NOT EVERY ENGINE HAS ALL THREE

POSTGRES has all three and will choose freely. **MYSQL** has nested loop and hash join (8.0.18+; from 8.0.20 hash join fully replaced the old block nested loop) — but **no merge join**. **SQLITE** implements **every join as a nested loop**, with no hash or merge join at all — which is exactly why an index on the join key matters more on SQLite than anywhere else.

■ Find the slow queries in the first place

POSTGRESQL

```
-- needs: shared_preload_libraries = 'pg_stat_statements'
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

SELECT calls, mean_exec_time, total_exec_time, query
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 20;
```

Sort by **total** time, not mean: a 5 ms query run 2 M times costs more than a 9 s report run nightly.

MYSQL

```
SELECT digest_text, count_star,
       avg_timer_wait/1e9 AS avg_ms,
       sum_timer_wait/1e9 AS total_ms
FROM performance_schema.events_statements_summary_by_digest
ORDER BY sum_timer_wait DESC
LIMIT 20;
```

Or enable the slow query log with `long_query_time = 1` and read it with `mysqldumpslow`.

Rule of thumb: **the fastest query is the one you never send**. Before tuning, ask whether the application needs those rows at all — caching, batching and pagination beat every index.

A B-tree index is a **sorted copy** of some columns plus a pointer back to the row. That single mental model explains everything below: you can seek to a prefix of the sort order, you can read ranges forward or backward, and you can't skip a leading column.

■ Composite index column order

An index on (a, b, c) serves WHERE a, WHERE a AND b, and WHERE a AND b AND c. It does **not** serve WHERE b alone — that's the **leftmost prefix rule**. One well-ordered composite index usually replaces three single-column ones.

THE ORDERING RULE: EQUALITY → SORT → RANGE

Put columns compared with = first, then the ORDER BY column, then columns compared with < > BETWEEN LIKE 'x%'. **A range column consumes the index:** nothing after it can be used for seeking.

WRONG ORDER — RANGE BEFORE SORT

```
-- query
WHERE tenant_id = ? AND created_at > ?
ORDER BY priority DESC

-- index (tenant_id, created_at, priority)
-- → seeks on tenant+created_at, then
-- → sorts every match by priority
```

SORT BEFORE RANGE

```
-- same query
WHERE tenant_id = ? AND created_at > ?
ORDER BY priority DESC

-- index (tenant_id, priority, created_at)
-- → rows arrive already in priority
-- → order; LIMIT can stop early
```

Which wins depends on selectivity — if created_at > ? eliminates 99.9 % of rows, the first index may still be better. **Measure both.** The rule is a starting hypothesis, not a law.

■ Covering indexes (index-only scans)

If the index contains **every column the query touches** — SELECT list, WHERE, ORDER BY — the engine never visits the table at all. Often a 5–10× win on wide tables.

ENGINE	HOW	CONFIRM IN THE PLAN
POSTGRES	CREATE INDEX ON orders (customer_id) INCLUDE (total); INCLUDE stores the payload in leaf nodes only — it can't be searched or sorted on, but it keeps the index narrow. PG 11+.	Index Only Scan + Heap Fetches: 0
MYSQL	Add the columns to the index. Secondary indexes already contain the primary key , so SELECT id is free.	Extra: Using index (not "Using index condition")
SQLITE	Add the columns to the index.	USING COVERING INDEX ...

POSTGRES GOTCHA: HEAP FETCHES

An index-only scan still checks the **visibility map** to know whether a row is visible to your transaction. On a table with recent writes those bits aren't set, so Postgres falls back to reading the heap and you lose the benefit — visible as a non-zero Heap Fetches. VACUUM sets those bits. A freshly bulk-loaded table that has never been vacuumed will quietly not use index-only scans.

■ What silently kills a seek (sargability)

An index works on the **bare column**. Wrap it in anything and the sort order no longer applies, so the engine scans.

THIS SCANS	THIS SEEKS	WHY
WHERE YEAR(created_at) = 2025	WHERE created_at >= '2025-01-01' AND created_at < '2026-01-01'	Function on the column. Half-open range also handles time components correctly.
WHERE LOWER(email) = ?	CREATE INDEX ON users (LOWER(email));	Index the expression itself. PG LT MY 8.0.13+. MySQL's default collation is already case-insensitive.

THIS SCANS	THIS SEEKS	WHY
WHERE name LIKE '%acme%'	pg_trgm GIN index PG , or full-text search	A leading wildcard has no prefix to seek to. LIKE 'acme%' is fine.
WHERE user_id = '42' (int column)	WHERE user_id = 42	Implicit cast. In MySQL a string-vs-number comparison can force a full scan and, worse, compare unexpectedly.
WHERE status != 'done'	WHERE status IN ('new', 'running')	Negation matches most rows, so a scan is genuinely cheaper. Enumerate what you want instead.
WHERE a = ? OR b = ?	SELECT ... WHERE a = ? UNION ALL SELECT ... WHERE b = ?	Two different columns can't share one seek. See §3.

■ When B-tree is the wrong index

B-tree is the default and the right answer ~90 % of the time. The exceptions are worth knowing, because no amount of column reordering fixes them.

TYPE	USE IT FOR	EXAMPLE	ENGINE
GIN	Many values inside one column: jsonb keys, arrays, full-text, and substring search via pg_trgm.	CREATE INDEX ON docs USING GIN (body jsonb_path_ops);	POSTGRES
BRIN	Huge naturally ordered append-only tables (time series). Stores min/max per block range — tiny index, but useless if the column isn't correlated with physical order.	CREATE INDEX ON events USING BRIN (created_at);	POSTGRES
GIST	Ranges, geometry, nearest-neighbour, exclusion constraints.	EXCLUDE USING GIST (room WITH =, during WITH &&)	POSTGRES
Full-text	Word search. Don't fake it with LIKE '%word%'.	tsvector + GIN PG · FULLTEXT MY · FTS5 LT	ALL

■ Indexes are not free

Every index is extra work on INSERT, UPDATE and DELETE, extra disk, and extra memory competing for cache. Two indexes with the same leading columns are usually one index too many.

```
-- unused indexes (PG)
SELECT relname, indexrelname, idx_scan,
       pg_size_pretty(pg_relation_size(indexrelid))
FROM   pg_stat_user_indexes
WHERE  idx_scan = 0
ORDER BY pg_relation_size(indexrelid) DESC;
```

Check uptime before trusting idx_scan = 0 — stats reset on pg_stat_reset(), and a quarterly report's index will look unused in May.

■ Partial indexes

Index only the rows you query. A job queue where status='pending' is 0.1 % of a 50 M row table gets an index that is **1000× smaller**, stays in cache, and costs nothing to maintain for the other 99.9 %.

```
CREATE INDEX jobs_pending_idx
ON jobs (created_at)
WHERE status = 'pending';    -- PG / SQLite
```

The planner only uses it when the WHERE clause provably implies the index predicate. MySQL has no partial indexes — the usual workaround is a generated column, or an index on (status, created_at).

BUILD WITHOUT LOCKING

CREATE INDEX CONCURRENTLY ... **PG** avoids holding a write lock. It's slower, can't run in a transaction, and can leave an INVALID index behind if it fails — check for that, drop, retry.

■ N+1 — the most expensive bug in ORMs

One query to fetch the list, then one query per item. Each is 1 ms and indexed, so nothing shows up in the slow query log — yet the endpoint takes 800 ms, almost all of it round-trip latency.

1 + 200 QUERIES

```
SELECT * FROM posts LIMIT 200;
-- then, per post, in application code:
SELECT * FROM users WHERE id = ?;
```

2 QUERIES

```
SELECT * FROM posts LIMIT 200;
SELECT * FROM users WHERE id = ANY(?);
-- or a single JOIN
```

Most ORMs have a name for the fix: `includes` (Rails), `selectinload` (SQLAlchemy), `Include` (EF Core), `with` (Laravel), `prefetch_related` (Django).

Tell: the same query shape repeated with different parameters in your query log.

■ OFFSET pagination collapses at depth

OFFSET 100000 makes the database produce 100 020 rows in order and throw away 100 000 of them. Page 1 is instant; page 5 000 is a table scan. Cost grows linearly with page number.

O(OFFSET) — SLOWER EVERY PAGE

```
SELECT id, title, created_at
FROM events
ORDER BY created_at DESC, id DESC
LIMIT 20 OFFSET 100000;
```

O(1) — KEYSET / "SEEK" PAGINATION

```
SELECT id, title, created_at
FROM events
WHERE (created_at, id) < (?, ?) -- last row of prev page
ORDER BY created_at DESC, id DESC
LIMIT 20;
```

Needs an index on `(created_at DESC, id DESC)` and a **tiebreaker column** — without `id`, rows sharing a timestamp get skipped or duplicated across pages. Row-value comparison works on [PG](#), [MYSQL 8.0.14+](#), [SQLITE 3.15+](#). Trade-off: no "jump to page 47" — which is why infinite scroll and "load more" are the natural UIs for it.

■ OR across columns → UNION ALL

```
-- one seek is impossible: scan
SELECT * FROM t WHERE a = ? OR b = ?;

-- two seeks, indexes on (a) and (b)
SELECT * FROM t WHERE a = ?
UNION ALL
SELECT * FROM t WHERE b = ? AND a <> ?;
```

The extra predicate avoids duplicates without the sort that `UNION (distinct)` would add. Postgres can sometimes do this itself via a `BitmapOr`; MySQL's `index_merge` is less reliable.

■ SELECT * is not free

- Wide columns (JSON, TEXT, BLOB) get read and sent for nothing — and in Postgres may mean extra TOAST fetches.
- It defeats **covering indexes**: one unneeded column forces a table lookup per row.
- It breaks silently when someone adds a column.

■ Counting large tables

`COUNT(*)` is exact and $O(n)$ — Postgres and InnoDB must visit every row for MVCC visibility. On 100 M rows that's seconds, per page load.

```
-- good enough for "About 4,300,000 results"
SELECT reltuples::bigint
FROM pg_class WHERE relname = 'events'; -- PG

-- exact count of a bounded window
SELECT count(*) FROM (
  SELECT 1 FROM events WHERE ... LIMIT 1001
) s; -- "1000+" — cheap and honest
```

`reltuples` is an estimate maintained by `VACUUM/ANALYZE`. For exact live counts, keep a counter table updated by trigger — and accept the write contention that comes with it.

■ DISTINCT is often a symptom

Reaching for `DISTINCT` to remove duplicates usually means a join fanned out rows. It hides the bug behind a sort of the whole result. Fix the join, or use `EXISTS`:

```
SELECT u.* FROM users u
WHERE EXISTS (SELECT 1 FROM orders o
              WHERE o.user_id = u.id);
-- stops at the first match per user
```

Batch your writes. 10 000 single-row `INSERTs` in autocommit means 10 000 transactions and 10 000 fsyncs. One multi-row `INSERT` — or one explicit transaction — is routinely **100× faster**. This is the single biggest win available on SQLite, and a large one everywhere else.

■ POSTGRESQL

Knob	What to know
<code>work_mem</code>	Per sort/hash node, per connection — not per query. A query with 4 sorts across 8 parallel workers can use many multiples. Raise it per-session for a big report rather than globally.
<code>shared_buffers</code>	Commonly ~25 % of RAM. Postgres also benefits from the OS page cache, so more is not automatically better.
<code>effective_cache_size</code>	Not an allocation — a hint about total cache available. Too low and the planner avoids index scans.
<code>random_page_cost</code>	Defaults assume spinning disks. On SSD/NVMe, lowering to 1.1 makes index scans correctly look cheaper.
<code>autovacuum</code>	Reclaims dead rows and sets visibility-map bits. Bulk loads outpace it — run ANALYZE manually after one. Never turn it off.

BLOAT & LONG TRANSACTIONS

An UPDATE writes a new row version; the old one lingers until vacuumed. An idle-in-transaction connection **blocks vacuum across the whole database**, so dead rows pile up and scans get slower for everyone. Check `pg_stat_activity` for state = 'idle in transaction' and set `idle_in_transaction_session_timeout`.

LIKE 'foo%' not using the index? In any non-C collation you need `CREATE INDEX ON t (col text_pattern_ops)`; — the default operator class sorts by locale rules, which don't match character-by-character prefix comparison. That index won't serve ordinary `< >` comparisons, so you may need both.

■ MYSQL / INNODB

EVERYTHING FOLLOWS FROM THE CLUSTERED PK

InnoDB stores the **table itself** in primary-key order, and every secondary index stores the primary key as its row pointer. Two consequences:

- **A fat PK bloats every secondary index.** A 36-char UUID string PK across 6 indexes wastes gigabytes versus a BIGINT.
- **A random PK fragments writes.** UUIDv4 inserts land in random pages, causing page splits and random I/O. Use `AUTO_INCREMENT`, or time-ordered **UUIDv7 / ULID** if you need opaque IDs.

Knob	What to know
<code>innodb_buffer_pool_size</code>	The one setting that matters most . Caches data and indexes. ~70 % of RAM on a dedicated server; the default is far too small.
<code>innodb_flush_log_at_trx_commit</code>	1 = fully durable (default). 2 = survives process crash but not power loss. Only relax it if you can afford to lose ~1 s of commits.
Prefix indexes	<code>INDEX (url(64))</code> keeps long-string indexes small — but such an index can never be covering and can't serve <code>ORDER BY</code> .

Read EXPLAIN right: type: ALL = full scan, index = full index scan (still every row!), range/ref/eq_ref/const = progressively better. Compare rows examined against rows returned — a big ratio is the whole problem in one number.

■ **SQLITE**

SQLite's defaults are tuned for **maximum compatibility and safety**, not speed. A handful of pragmas at connection time typically buys an order of magnitude.

```
PRAGMA journal_mode = WAL;      -- once per DB, persists
PRAGMA synchronous = NORMAL;    -- per connection
PRAGMA busy_timeout = 5000;     -- ms; default 0 (!)
PRAGMA foreign_keys = ON;       -- OFF by default (!)
PRAGMA cache_size = -64000;     -- negative = KiB (~64 MB)
ANALYZE;                        -- planner needs stats
```

PRAGMA	WHY
journal_mode=WAL	Readers stop blocking the writer and vice versa. QoreDB opens SQLite in WAL by default. Adds -wal/-shm sidecar files — don't delete them.
busy_timeout	Default 0 means a concurrent writer fails instantly with SQLITE_BUSY instead of waiting. Most "SQLite can't handle concurrency" reports are this.
PRAGMA optimize	Run before closing a long-lived connection; refreshes stats cheaply.

SYNCHRONOUS = NORMAL: READ THIS BEFORE SHIPPING
In WAL mode, NORMAL **will not corrupt your database** — integrity is guaranteed. But it **does give up durability**: a power loss or OS crash can roll back recent committed transactions. Fine for caches, derived data and dev. For a system of record where a lost commit matters, keep FULL.

One writer at a time, always. If two processes must write concurrently and busy_timeout isn't enough, SQLite is the wrong tool — not a tuning problem.

■ **The 10-minute triage**

In order. Most slow queries are fixed by step 3.

- ☐ **Reproduce with numbers.** Get the actual query and its parameters. "The dashboard is slow" is not a query.
- ☐ **Get the plan.** EXPLAIN ANALYZE. Read bottom-up.
- ☐ **Compare estimated vs actual rows.** Off by >10×? Run ANALYZE and re-plan before touching anything else.
- ☐ **Find the biggest scan.** Which node discards the most rows? That filter is your index candidate.
- ☐ **Check sargability.** Is the column wrapped in a function or cast? (§2)
- ☐ **Order the index: equality → sort → range.** Try to make it covering.
- ☐ **Re-measure.** Same plan? The index isn't usable — find out why rather than adding another.
- ☐ **Only now touch config.** Memory settings fix a class of queries; they rarely fix this one.

DO THIS IN QOREDB
Everything on these four pages works in any client — that's the point. QoreDB just removes the tab-switching: **execution plan visualisation**, one editor across PostgreSQL, MySQL, SQL Server, SQLite, DuckDB, MongoDB, Redis, CockroachDB and ClickHouse, and **notebooks** (.qnb) that mix SQL and Markdown — so a triage like the one above becomes a runbook you can re-run and share instead of a scrollbar you lose.

Verify against your version. Behaviour and defaults change between releases. Primary sources: postgresql.org/docs · dev.mysql.com/doc · sqlite.org/docs.html.